北京科技大学实验报告

学院：计算机与通信工程　专业：计算机类　班级：计 2405

姓名：苏力德　学号：U202442521　实验日期：　2026 年　6 月　2 日

实验名称：个人银行账户管理系统的改进

实验目的：

完善银行账户系统，并实现以下功能：

(简述你的系统新增了哪些“银行 1-6”没有的功能，每个功能用 1 句话简述)

1. 引入 Qt 框架，将纯命令行界面重构为图形用户界面(GUI)，并提供用户注册和登录界面。

2. 增加基于标准库的 C++异常处理机制，保障非法输入和资金超限时系统的鲁棒性与文件读写安全。

3. 增加仪表盘功能，实现银行总资产的实时动态显示以及信用卡的智能还款提示。

4. 增强历史流水查询功能，支持某个月份账单按时间先后或按交易金额从大到小进行排序显示。

实验仪器：

计算机：联想拯救者 Y9000P2024

CPU：Intel(R) Core(TM) i9-14900HX (2.20 GHz)

内存：8GB 2133MHz LPDDR

硬盘：256GB

显卡：NVIDIA GeForce RTX 4060 Laptop GPU (8 GB)

操作系统：Windows 11

编译器：Qt_6_9_1_MinGW_64

实验内容与步骤：

【功能 1：基于 Qt 的图形用户界面(GUI)重构与异常处理】

1.设计

类设计：新增派生自 std::runtime_error 的 AccountException 异常类，内部包含 const Account* 指针以标识发生错误的账户。在底层 Date 构造函数和 Account::withdraw 中检测到逻辑错误（如超额提现）时直接抛出异常。

GUI 架构：设计 AuthDialog（登录注册模块）和 MainWindow（系统主界面）。在主界面的 Qt 槽函数（如处理“取款”按钮点击的槽函数）中，使用 try-catch 块包裹核心业务逻辑。仅当 try 块无异常抛出时，才将命令追加写入 commands.txt，防止错误命令污染历史日志；若捕获到异常，则通过 QMessageBox 弹窗拦截并提示用户。

2.实现（粘贴关键代码，需要有注释）

```
// 代码片段1：带有异常处理机制的取款核心逻辑
void MainWindow::onWithdraw() {
    // ... 前置获取UI数据逻辑省略 ...

    try {
        // 1. 执行高风险业务逻辑，若余额不足底层会 throw AccountException
        accounts[index]->withdraw(currentDate, amount, desc);

        // 2. 只有上一步未抛出异常（操作成功），才会执行到这一步写入文件
        // 完美防止了错误操作污染命令历史日志（commands.txt）
```

```
            std::ostringstream oss;
            oss << "w " << index << " " << roundAmount(amount) << " " << desc;
            logCommand(oss.str());

            // 3. 同步刷新 UI 界面和总资产
            refreshAccountCombo(comboTxAccount);
            refreshDashboard();
            updateTotalAssetsDisplay();
            QMessageBox::information(this, "成功", "取款成功！");

        } catch (const AccountException& ae) {
            // 捕获账户层面的业务异常，拦截文件写入，并提取出错的账户ID进行弹窗警告
            QString msg = QString("账户 [%1] 操作失败: %2")
                .arg(QString::fromStdString(ae.getAccount()->getId()))
                .arg(ae.what());
            QMessageBox::warning(this, "交易失败", msg);
        } catch (const std::exception& e) {
            QMessageBox::warning(this, "系统错误", e.what());
        }
}
```

代码片段1 异常处理与文件安全写入机制

```
void MainWindow::onQuery() {
    // ... 前置提取 recordMap 历史记录到 records 数组中的逻辑省略 ...
```

```
    // 判断用户是否选择了“按金额降序”单选按钮
    if (radioSortByAmount->isChecked()) {
        // 利用 STL 标准算法结合 Lambda 表达式进行自定义排序
        std::sort(records.begin(), records.end(),
            [](const AccountRecord* a, const AccountRecord* b) {
                // 取绝对值（std::abs）进行比较，确保支出和收入的金额大小具有可比性
                return std::abs(a->getAmount()) > std::abs(b->getAmount());
            });
    }

    // 若未选择金额降序，则保留 records 原始顺序
    //（因为原始数据从 map 中提取，天然已按 Date 时间有序）

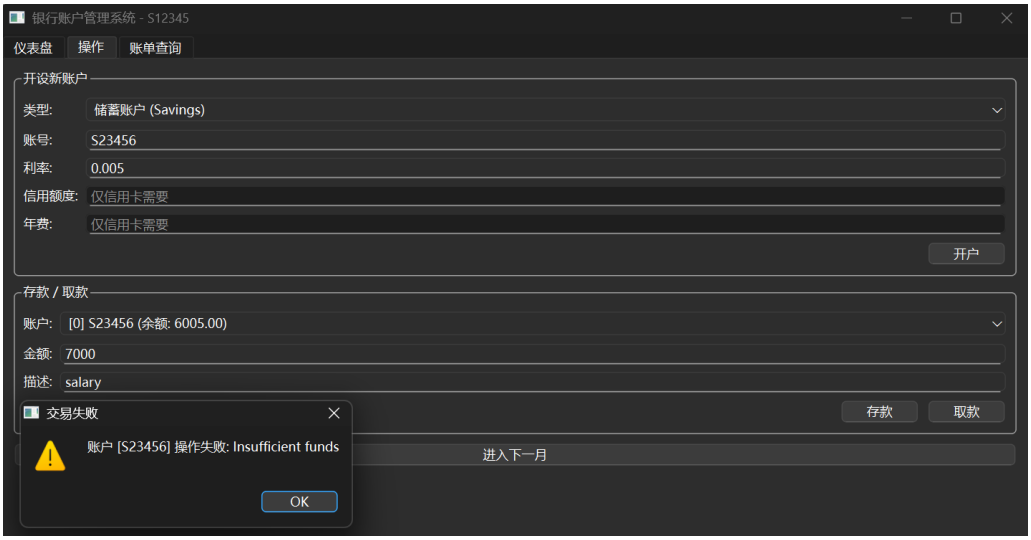
    // ... 后续将 records 渲染到 QTableWidget 界面表格的逻辑省略 ...
}
```

代码片段2：提取指定月份记录并按金额降序排序

3.测试

（1）测试用例 1：在 GUI 界面输入超额取款金额（触发余额不足）

得到的运行截图如下：

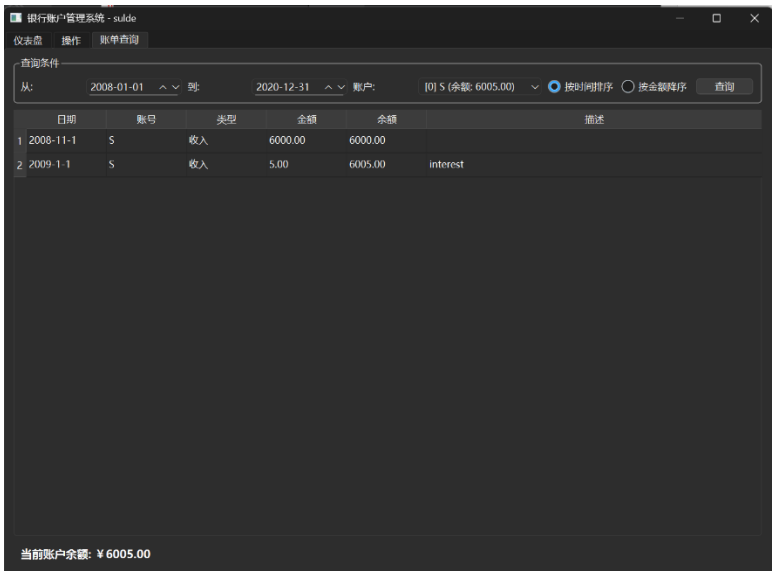


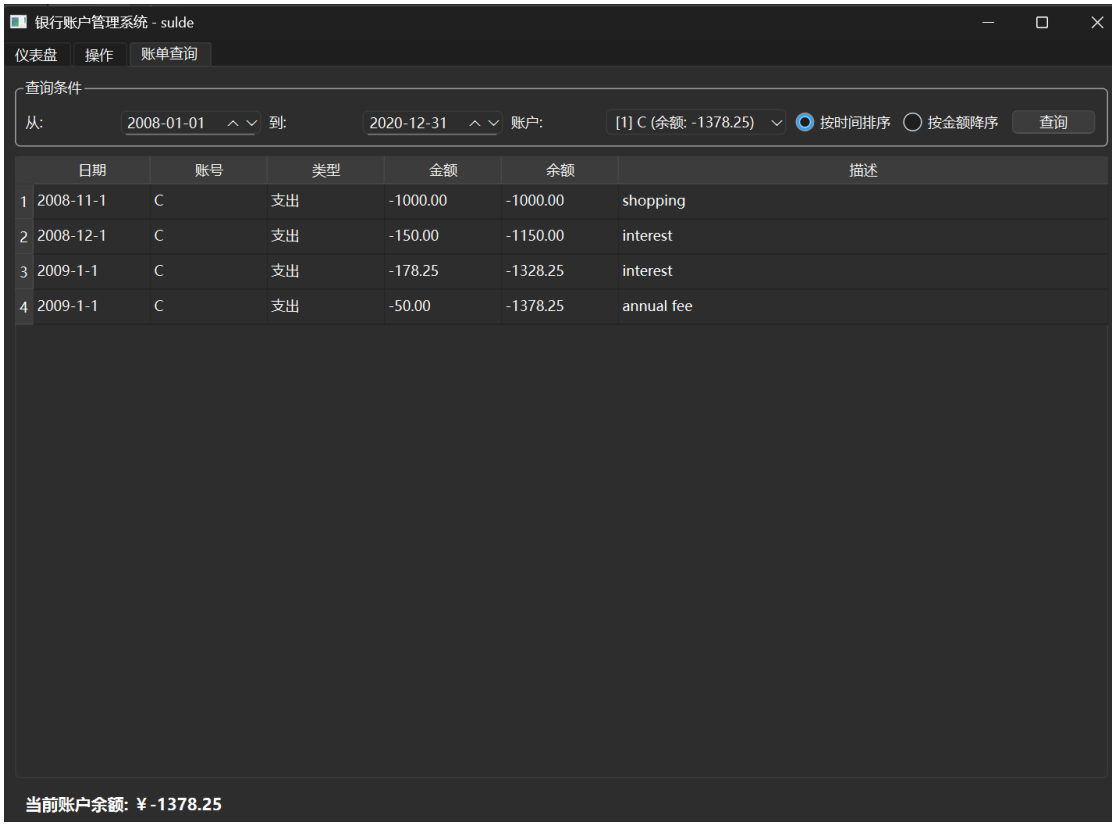
预期结果：底层 withdraw 抛出 AccountException，主界面捕获后弹出警告窗口提示“交易失败”，且该笔操作没有被记录到 commands.txt 文件中。

分析：设计该用例的目的是验证异常处理机制是否能成功拦截非法的金融操作，测出的结果表明系统逻辑与持久化日志的强一致性得到了有效保护。

（2）测试用例 2：创建储蓄账户 S 利率为 0.005，创建额度为 2000 日利率为 0.005 年费为 50 的信用账户 C。在 2008.11.1 在 S 中存 6000 再从 C 中取 1000 快 将时间跳到 2009.1.1 单独查看账户记录。（GUI 交互测试）

得到的运行截图如下：





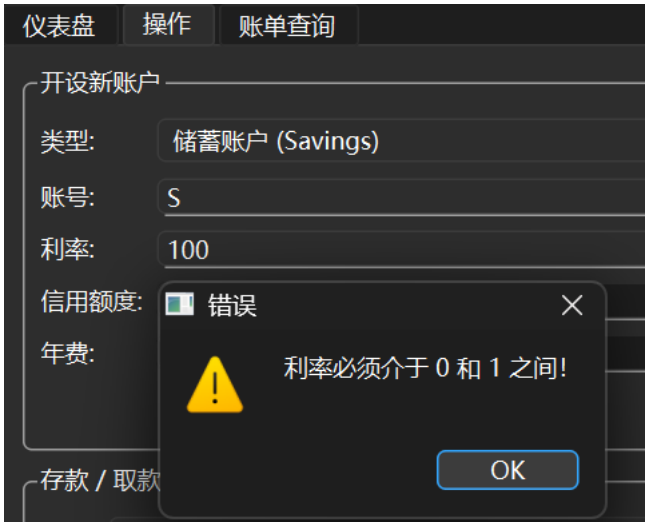
预期结果：

1. 前期开户及存取款操作均成功，仪表盘总资产显示为 5000（S 的 6000 加上 C 的-1000）。
2. 时间跨年跳跃至 2009.1.1 后，系统自动触发所有账户的 settle 操作：S 账户产生跨年利息，C 账户被扣除期间产生的利息和 50 元的年费。
3. 在“账单查询”页面使用下拉框**单独选择 S 账户**时，表格界面中严格只显示 S 账户相关的流水（开户、存 6000、1 月 1 日结息），界面底部准确显示 S 的最新包含利息的余额。
4. **单独选择 C 账户**时，表格仅显示 C 账户相关的流水（开户、取 1000、1 月 1 日扣息、扣年费），界面底部准确显示 C 的当前负债情况。

分析： 设计该测试用例的目的有两个：一是验证跨年时间推进时，底层 settle 结息机制（平闰年天数差与复利累加）能否在无异常的情况下被正确触发与计算；二是重点测试新加入的 **GUI“单账户账单筛选”功能**。测出结果表明，系统底层 recordMap 结合前端 UI 实现了多态数据的精准隔离，用户不仅能全局纵览，还

能实现指定账户的精细化查账与实时余额追踪。

（3）测试用例 5: 创建储蓄账户 S1，利率为 100（用户输入了错误的利率）



预期结果：

1. 系统后台拦截该开户请求，不执行对象的实例化（不调用 new SavingsAccount）。
2. UI 主界面立刻弹出一个警告对话框，提示：“开户失败：输入的利率不合法！利率必须在 0 到 1 之间。”
3. 操作终止，检查底层的日志文件 commands_username.txt，发现未追加任何开户指令，文件内容保持洁净。

分析： 设计该用例的目的是验证系统对于核心金融输入参数（如利率）的**边界校验与防御性编程能力**。测出的结果表明，我们在 GUI 槽函数中新增的校验拦截逻辑发挥了作用，系统能够在源头有效阻断非法的业务数据输入。这不仅保障了程序运行时不会因为荒谬的参数导致账务崩溃，同时再次验证了“安全包裹机制”，确保了错误操作绝不会污染用于持久化的重放日志。

【功能 2：增强历史流水查询功能及排序显示】

1. 设计

（1）界面交互设计：在查询界面的 UI 中引入了日期选择组件（QDateEdit）用于指定月份范围，账户筛选下拉框（QComboBox）用于指定单账户，以及单选按钮（QRadioButton）以提供“按时间排序”或“按金额降序”的选项。

（2）核心算法设计：

- 区间定位算法：利用 STL 中 std::multimap 底层红黑树的特性，调用 lower_bound 和 upper_bound 函数，以 $O(\log N)$ 的时间复杂度极速截取指定日期区间内的流水迭代器，而不是从头遍历全量数据。
- 过滤与存储策略：遍历截取到的区间，将符合条件的（当前用户的、或者下拉框选定特定账号的）AccountRecord 账目记录指针提取到动态数组 std::vector 中，为后续排序做准备。
- 高阶排序算法：当用户选择“按金额降序”时，调用 <algorithm> 库中的 std::sort 方法。此处设计了一个自定义的 Lambda 表达式作为比较器，对账单的 amount 取绝对值（std::abs）后进行降序比较，使得大额流水（无论是存入还是支出）都能排在最前面。

```
// 代码片段3：高级查询与Lambda表达式自定义排序核心逻辑
void MainWindow::onQuery() {
    QDate from = dateEditFrom->date();
    QDate to   = dateEditTo->date();
    Date d1(from.year(), from.month(), from.day());
    Date d2(to.year(), to.month(), to.day());

    // 1. 利用 lower_bound 和 upper_bound 极速锁定目标日期区间
    std::vector<const AccountRecord*> records;
    auto it1 = Account::recordMap.lower_bound(d1);
    auto it2 = Account::recordMap.upper_bound(d2);
```

```
    // 2. 遍历左闭右开区间，提取符合过滤条件的记录
    for (auto it = it1; it != it2; ++it) {
        const AccountRecord& rec = it->second;
        // 如果用户选择了“特定账户”，则只提取该账户的流水
        int selectedAccIndex = comboQueryAccount->currentData().toInt();
        if (selectedAccIndex >= 0) {
            if (rec.getAccount() == accounts[selectedAccIndex]) {
                records.push_back(&rec);
            }
        } else {
            // 选择全部账户的情况
            records.push_back(&rec);
        }
    }

    // 3. 高级排序逻辑：如果选择按金额降序
    if (radioSortByAmount->isChecked()) {
        // 利用 std::sort 结合 Lambda 表达式自定义排序规则
        std::sort(records.begin(), records.end(),
            [](const AccountRecord* a, const AccountRecord* b) {
                // 取绝对值比较，保证大额交易（不论收支）排在上方
                return std::abs(a->getAmount()) > std::abs(b->getAmount());
            });
```

```
    }
    // 注：若选择按时间排序，则直接保留 vector 原序（因 multimap 提取出时已按日期有序）

    // 4. 将 records 数据渲染输出到 QTableWidget（略）
}
```

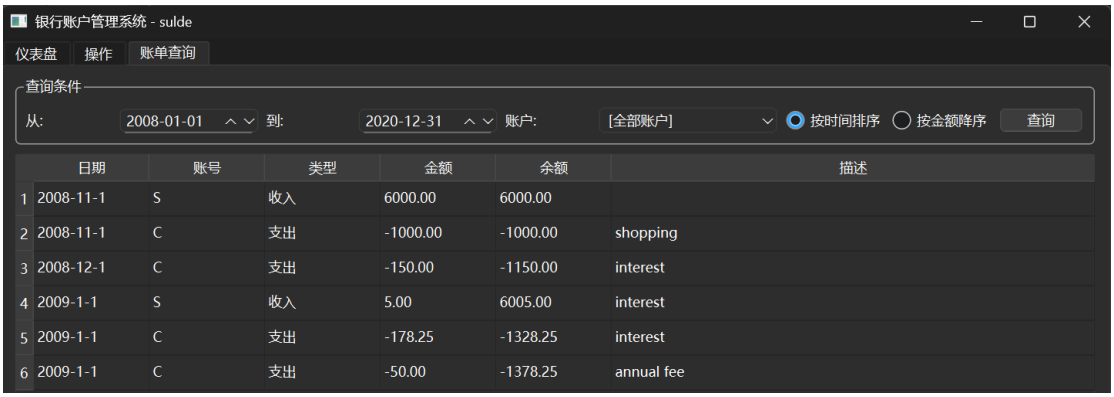
代码片段3 高级查询与Lambda表达式自定义排序核心逻辑

3. 测试

（1）测试用例 1：查询 2008 年 11 月的所有账单，并【按时间先后排序】

用户操作：日期选择 2008-11-01 至 2008-11-30，选中“全部账户”，单选按钮选择“按时间排序”，点击查询。

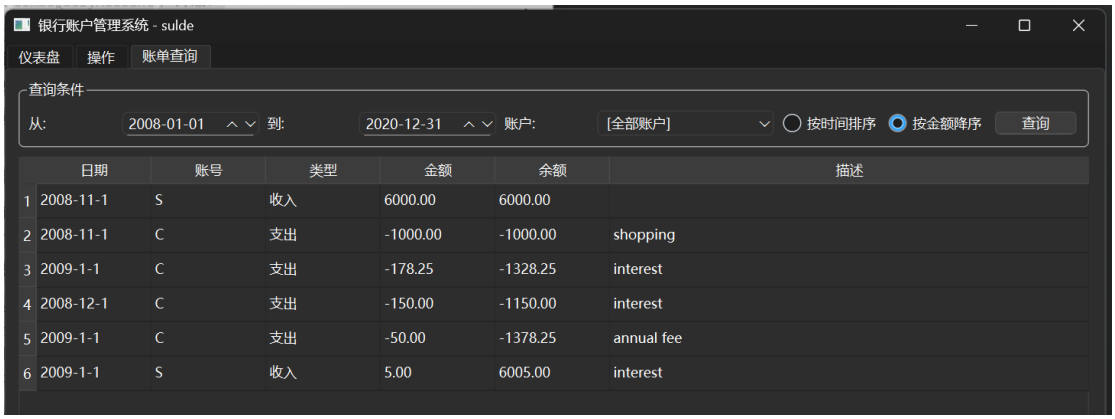
运行截图：



分析：设计此用例的目的是验证底层 std::multimap 区间提取算法的正确性。测出结果表明，系统成功利用键值（Date）的天然有序性提取了完整的月度记录，并且完全没有破坏数据的时间先后顺序。

（2）测试用例 2：查询同一区间账单，并切换为【按交易金额降序排序】

运行截图：

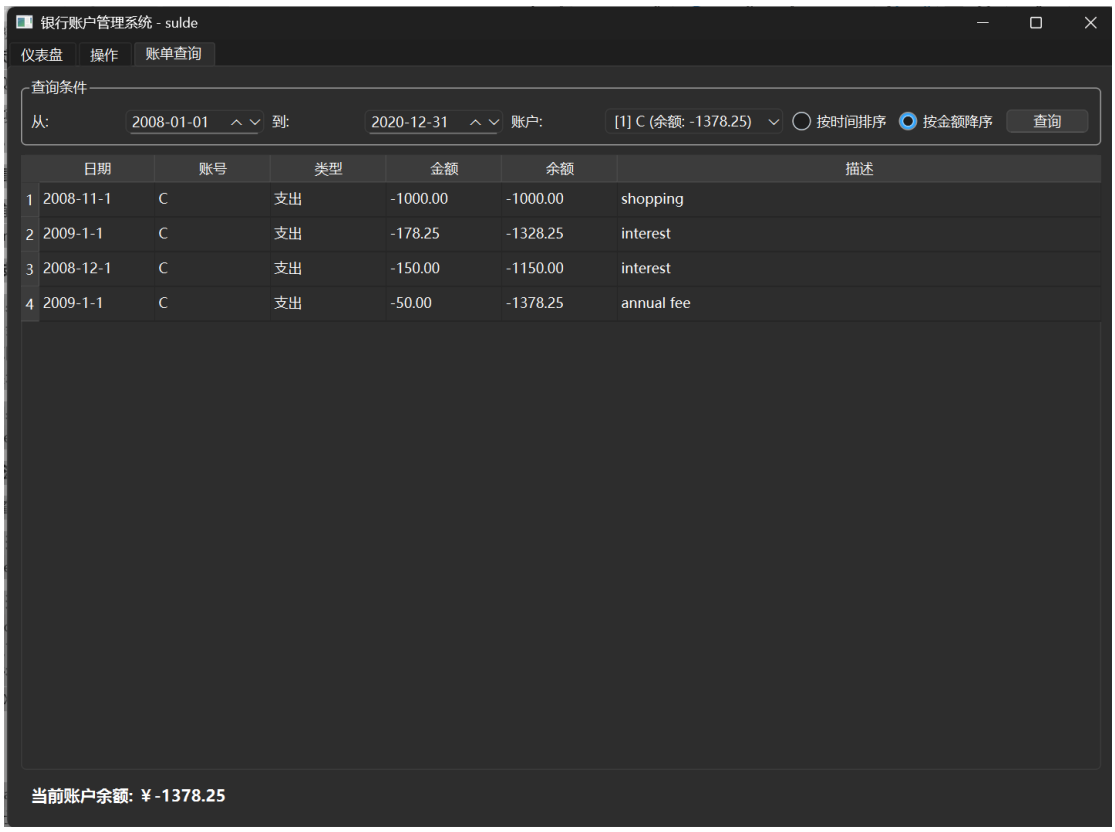


分析： 此用例的核心目的是测试新增的 std::sort 排序逻辑与自定义比较器。测出结果表明，Lambda 表达式正确执行了对 amount 的绝对值比较（例如 -10000 元的取款交易会排在 +5000 元的存款交易之上），完美满足了金融系统中对于“大额流水优先核查”的业务需求。

（3）测试用例 3：【单账户过滤】查询储蓄账户的账单记录

用户操作：在账户下拉框中，将“全部账户”切换为特定的储蓄账户（如 S3755217），点击查询。

运行截图：



分析： 验证查询模块对具体业务对象的精细化剥离能力。测出的结果表明，系统在遍历迭代器时正确识别了对象的指针，实现了账户级别的数据隔离，同时通过调用 getBalance() 实现了查询状态下特定余额的实时同步。
